Lab Session 6:3D Medical Image Registration- Brain MRI

2. Affine 3D Registration- Dipy:

- **Perform affine registration of the subject volume to the atlas volume using Dipy.**

- **Save the registered volume in NIfTI format as registered affine patient.nii.**

```
# Initial resampling for visualization
identity = np.eye(4)
affine_map = AffineMap(identity,
                       template_data.shape, template_affine,
                       moving_data.shape, moving_affine)
resampled = affine_map.transform(moving_data)

# Overlay slices
regtools.overlay_slices(template_data, resampled, None, 0, "Template", "Moving")
plt.show()
regtools.overlay_slices(template_data, resampled, None, 1, "Template", "Moving")
plt.show()
regtools.overlay_slices(template_data, resampled, None, 2, "Template", "Moving")
plt.show()

# Affine registration setup
nbins = 32
sampling_prop = None
metric = MutualInformationMetric(nbins, sampling_prop)
level_iters = [10, 10, 5]
sigmas = [3.0, 1.0, 0.0]
factors = [4, 2, 1]

affreg = AffineRegistration(metric=metric, level_iters=level_iters,
                            sigmas=sigmas, factors=factors)
```

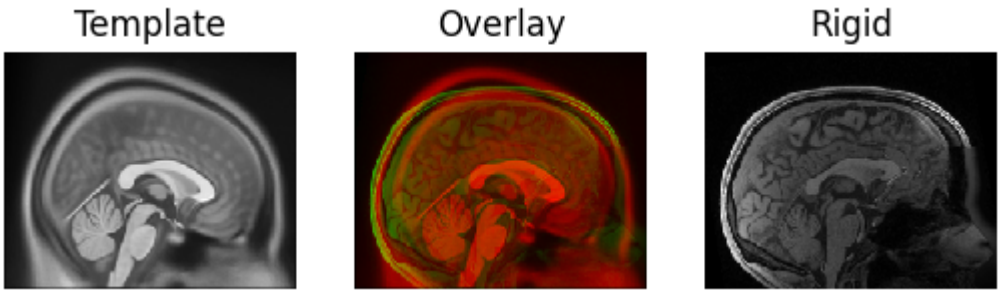
```
# Visualize translation result
transformed = translation.transform(moving_data)
regtools.overlay_slices(template_data, transformed, None, 0, "Template", "Translated")
plt.show()
regtools.overlay_slices(template_data, transformed, None, 1, "Template", "Translated")
plt.show()
regtools.overlay_slices(template_data, transformed, None, 2, "Template", "Translated")
plt.show()

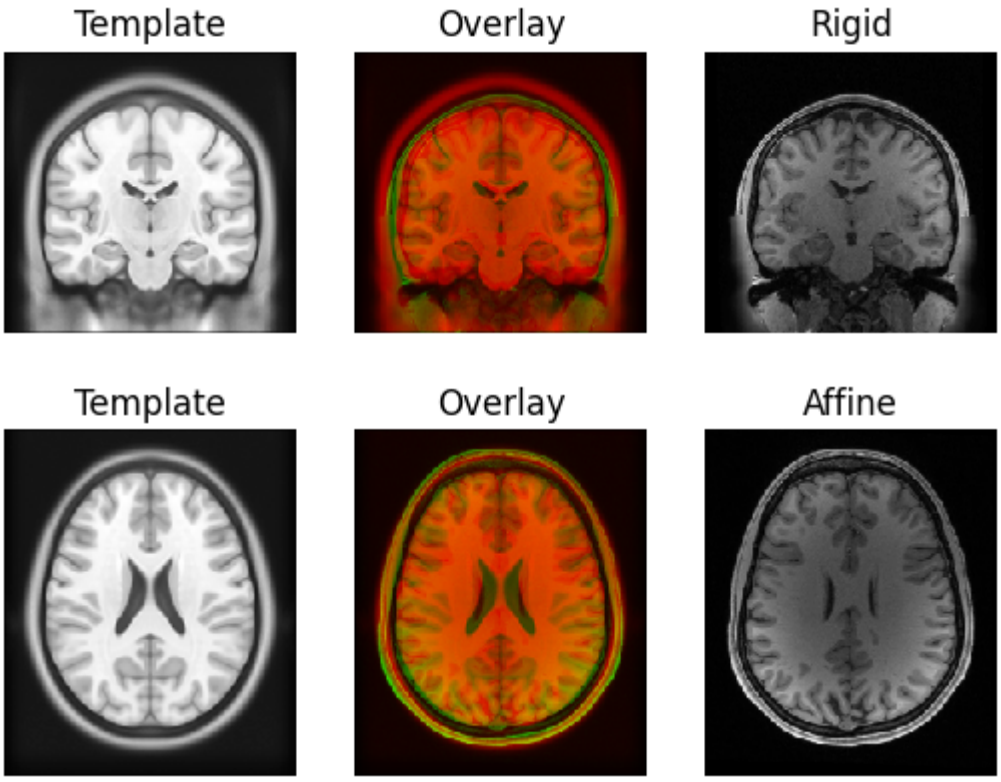
# Rigid-body registration
transform = RigidTransform3D()
rigid = affreg.optimize(template_data, moving_data, transform, params0,
                        template_affine, moving_affine,
                        starting_affine=translation.affine)

# Visualize rigid-body result
transformed = rigid.transform(moving_data)
regtools.overlay_slices(template_data, transformed, None, 0, "Template", "Rigid")
plt.show()
regtools.overlay_slices(template_data, transformed, None, 1, "Template", "Rigid")
plt.show()
regtools.overlay_slices(template_data, transformed, None, 2, "Template", "Rigid")
plt.show()

# Full affine registration
transform = AffineTransform3D()
affreg.level_iters = [1000, 1000, 100]
```

• Compare the registration in 2D using different plots. Create a figure with one slice (axial, sagittal, or coronal) of the original volume of the atlas, and how they overlap. Create a second figure with the same slice, but compare the registered volume, the atlas, and how they overlap.




Template
Overlay
Rigid
Template
Overlay
Affine

Deformable 3D Registration- Dipy:

- **Perform non-linear registration of the subject volume to the atlas volume using Dipy.**

- **Save the registered volume in NIfTI format as deformed patient.nii.**

- **Compare the registration in 2D using different plots. Create a figure with one slice (axial, sagittal, or coronal) of the original volume of the atlas, and how they overlap. Create a second figure with the same slice, but compare the registered volume and the atlas and how they overlap**

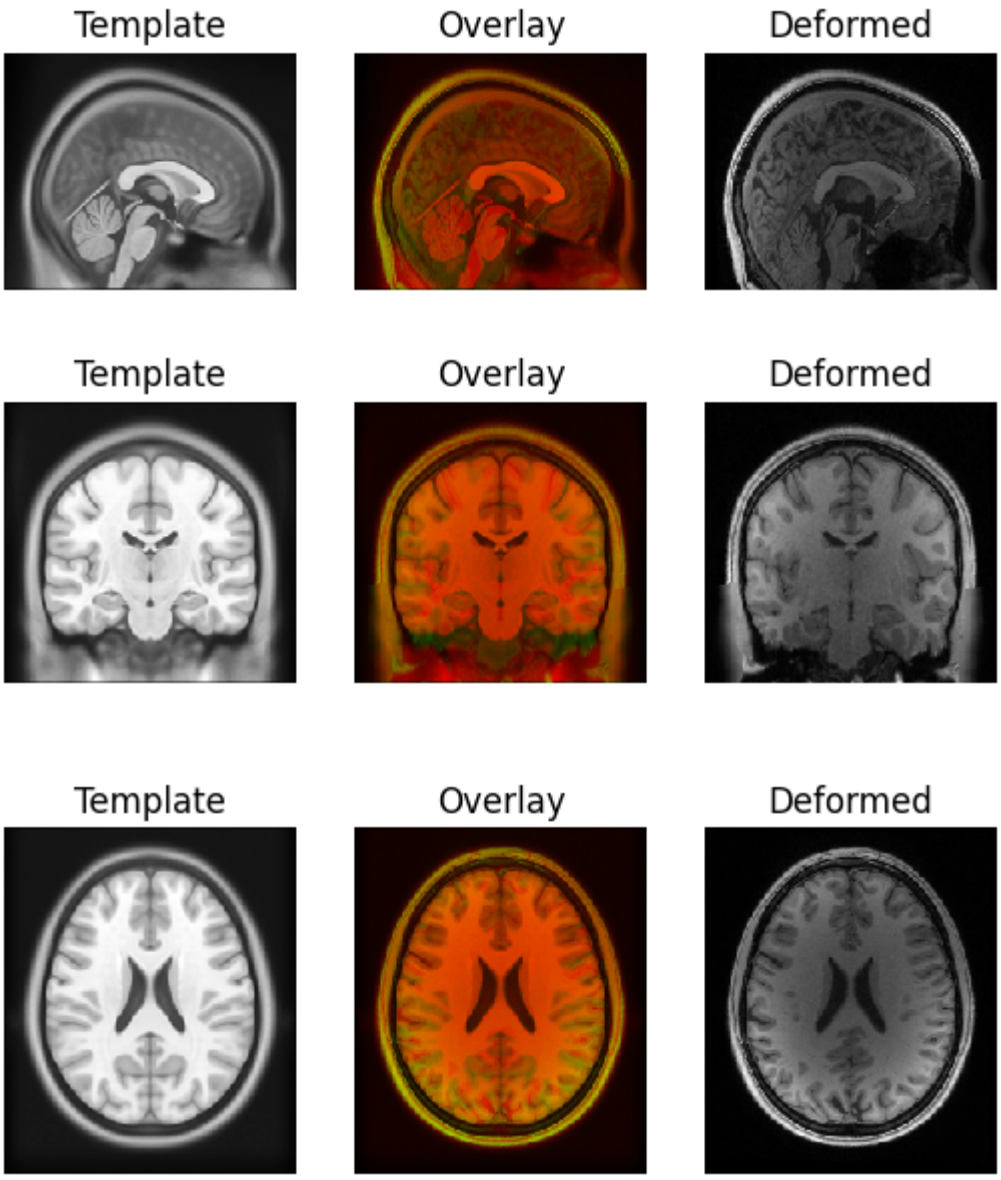
```
# Deformable registration setup
metric = CCMetric(3)  # Cross-Correlation Metric
level_iters = [10, 10, 5]  # Multi-resolution iterations
sdr = SymmetricDiffeomorphicRegistration(metric, level_iters)

# Perform deformable registration
mapping = sdr.optimize(template_data, moving_data, template_affine,
                       moving_affine, affine.affine)

# Apply the deformation field
warped_moving = mapping.transform(moving_data)

# Visualize deformable result
regtools.overlay_slices(template_data, warped_moving, None, 0, "Template", "Deformed")
plt.show()
regtools.overlay_slices(template_data, warped_moving, None, 1, "Template", "Deformed")
plt.show()
regtools.overlay_slices(template_data, warped_moving, None, 2, "Template", "Deformed")
plt.show()

# Save the deformed volume
deformed_img = nib.Nifti1Image(warped_moving, template_affine)
nib.save(deformed_img, "deformed_patient.nii")
```


Template
Overlay
Deformed
Template
Overlay
Deformed
Template
Overlay
Deformed

4. Registration with Segmented Data- Dipy:

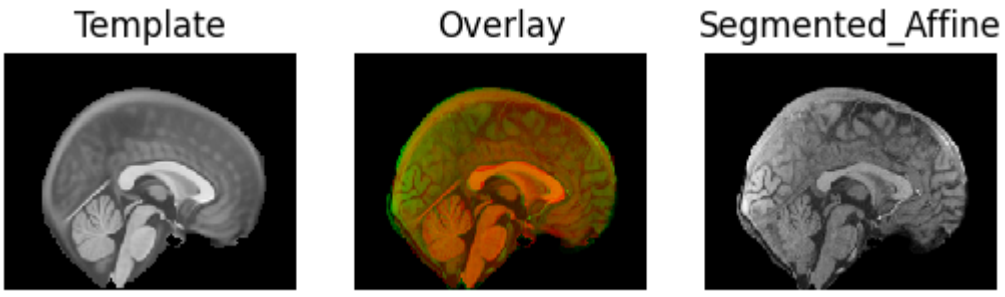
• Perform Affine and Non-linear registration of the segmented subject volume to the segmented atlas volume using Dipy.

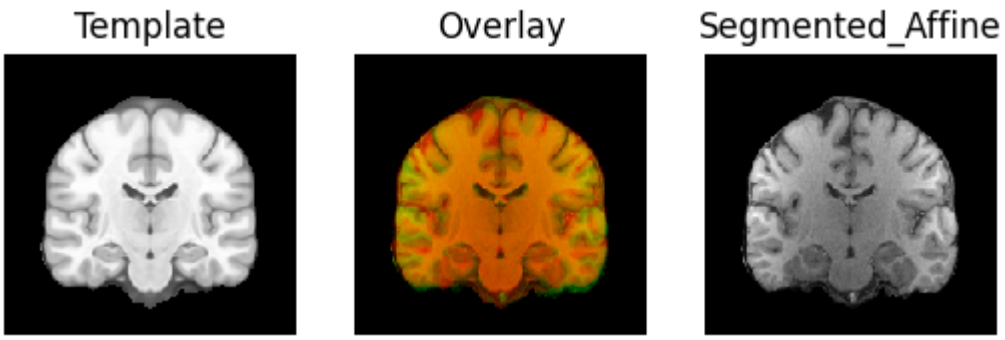
For this part, I copied the code from the previous part, but changed the files referring to the segmented atlas and patient this time.

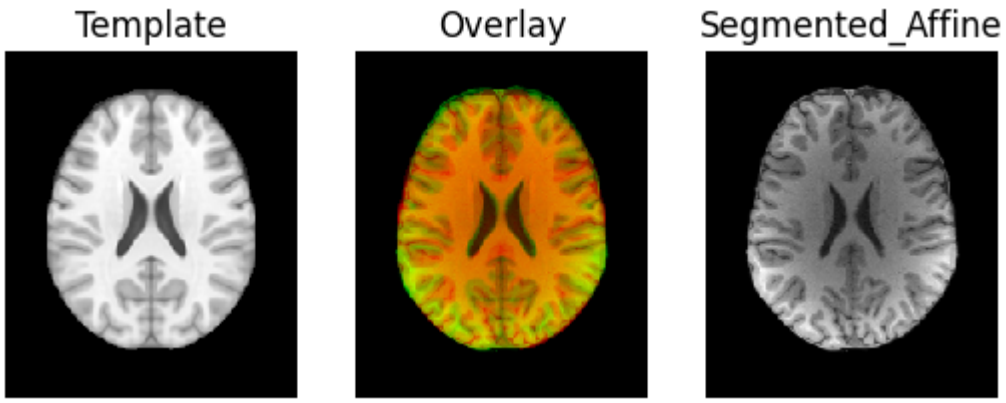
• Save the registered volumes in NIfTI format as segmented_deformed_ patient.nii, segmented_affine_patient.nii

• Compare the registration in 2D using different plots. Create a figure with one slice (axial, sagittal, or coronal) of the original volume of the atlas, and how they overlap. Create a second figure with the same slice, but compare the registered volume and the atlas and how they overlap

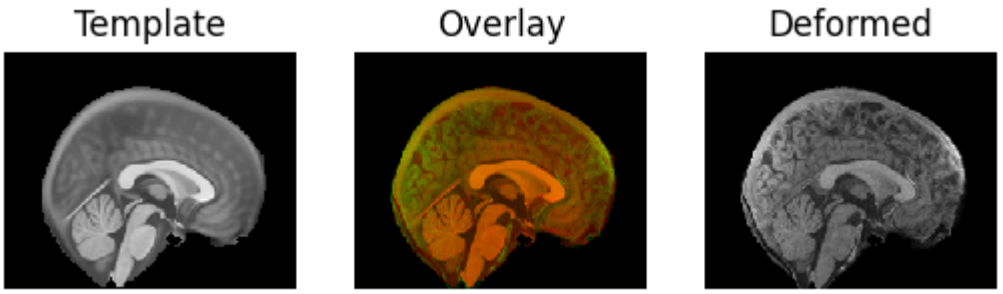
Affine segmented registration:

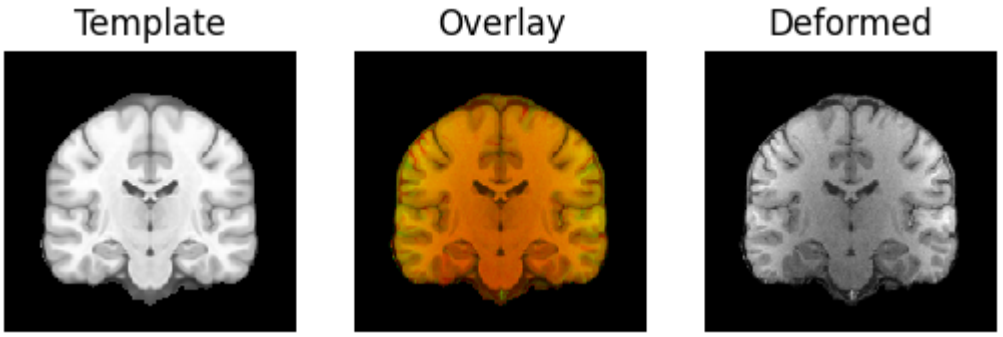


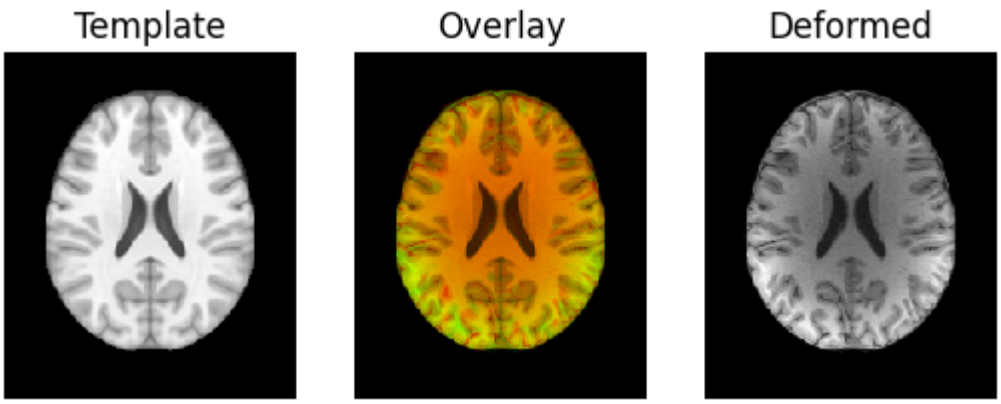




Deformed segmented registration:



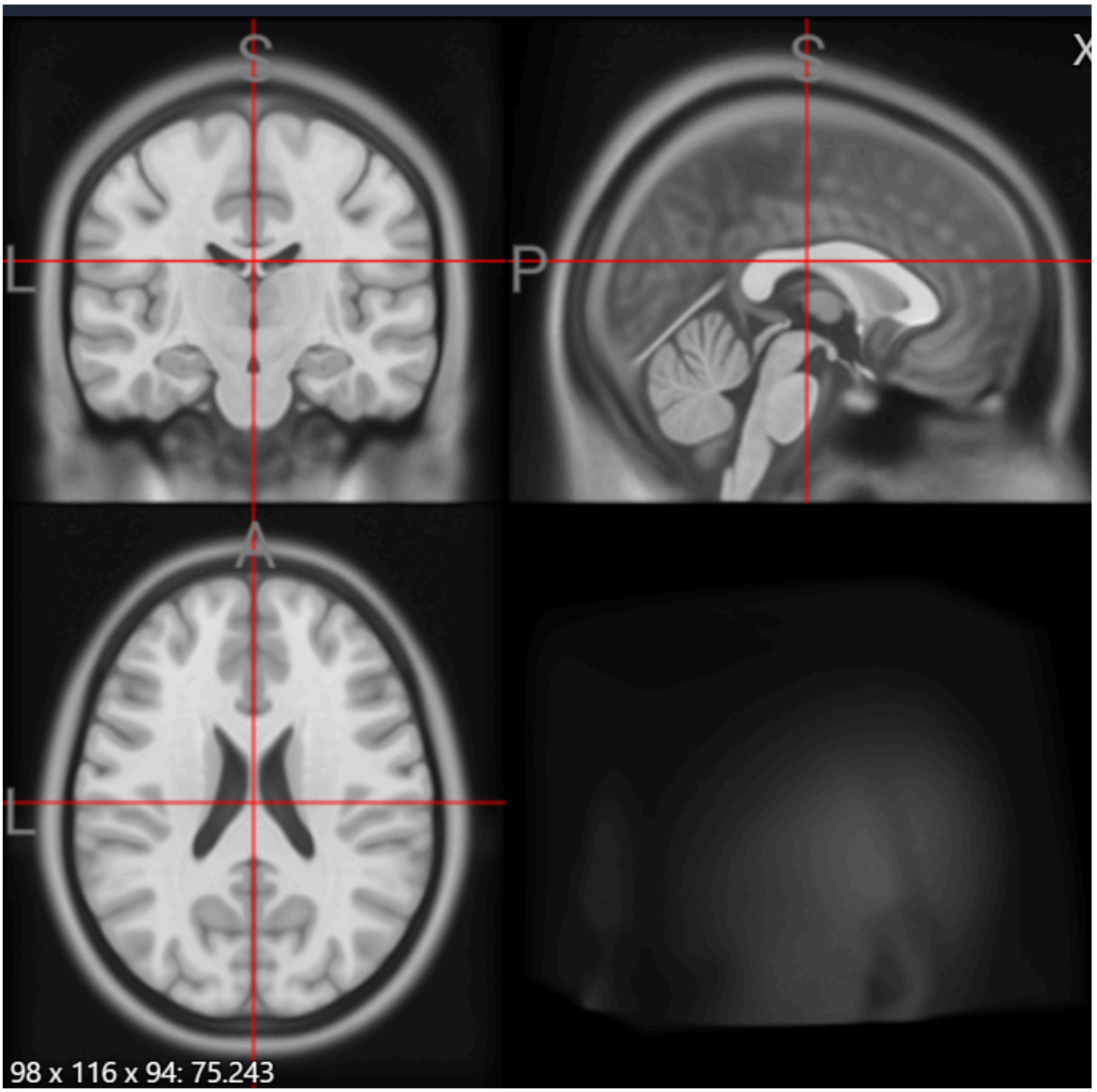



Template
Overlay
Deformed

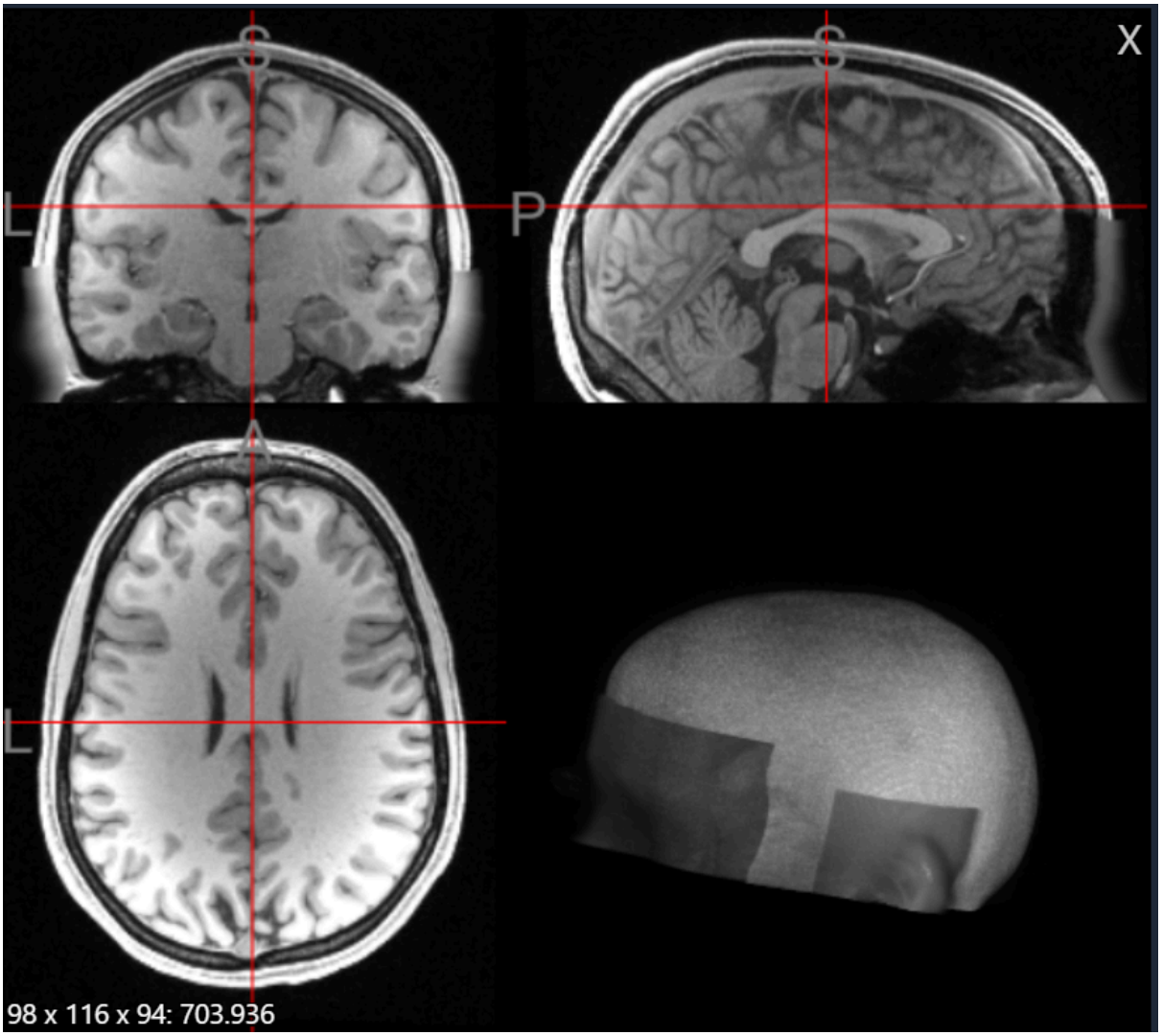
5. Report- Visualization and Analysis:

• Provide a screenshot of 3DSlicer where you compare the registered volumes and the respective atlas volumes.

atlas_not_segmented:

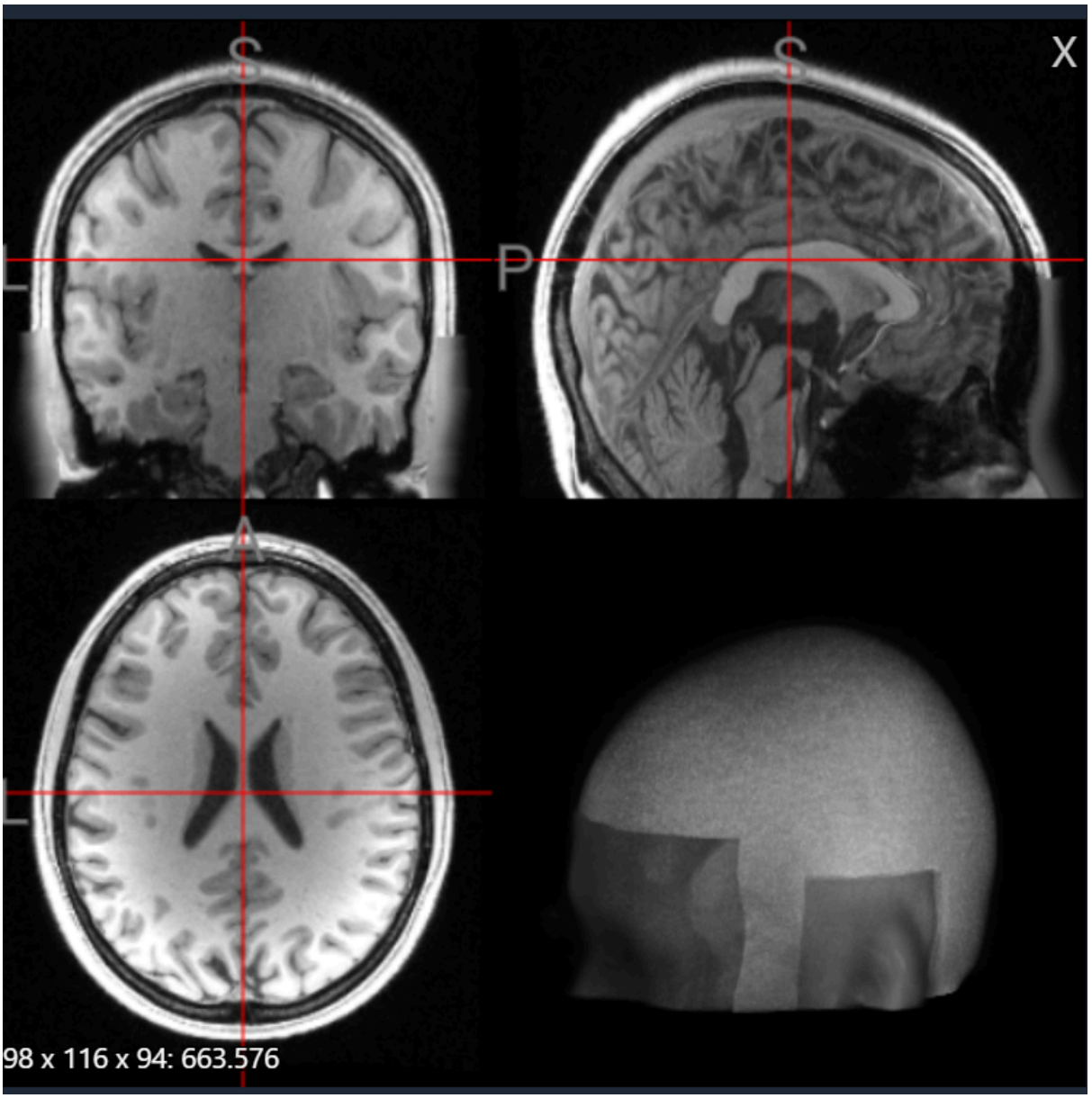


registered_affine_patient:



deformed_patient:

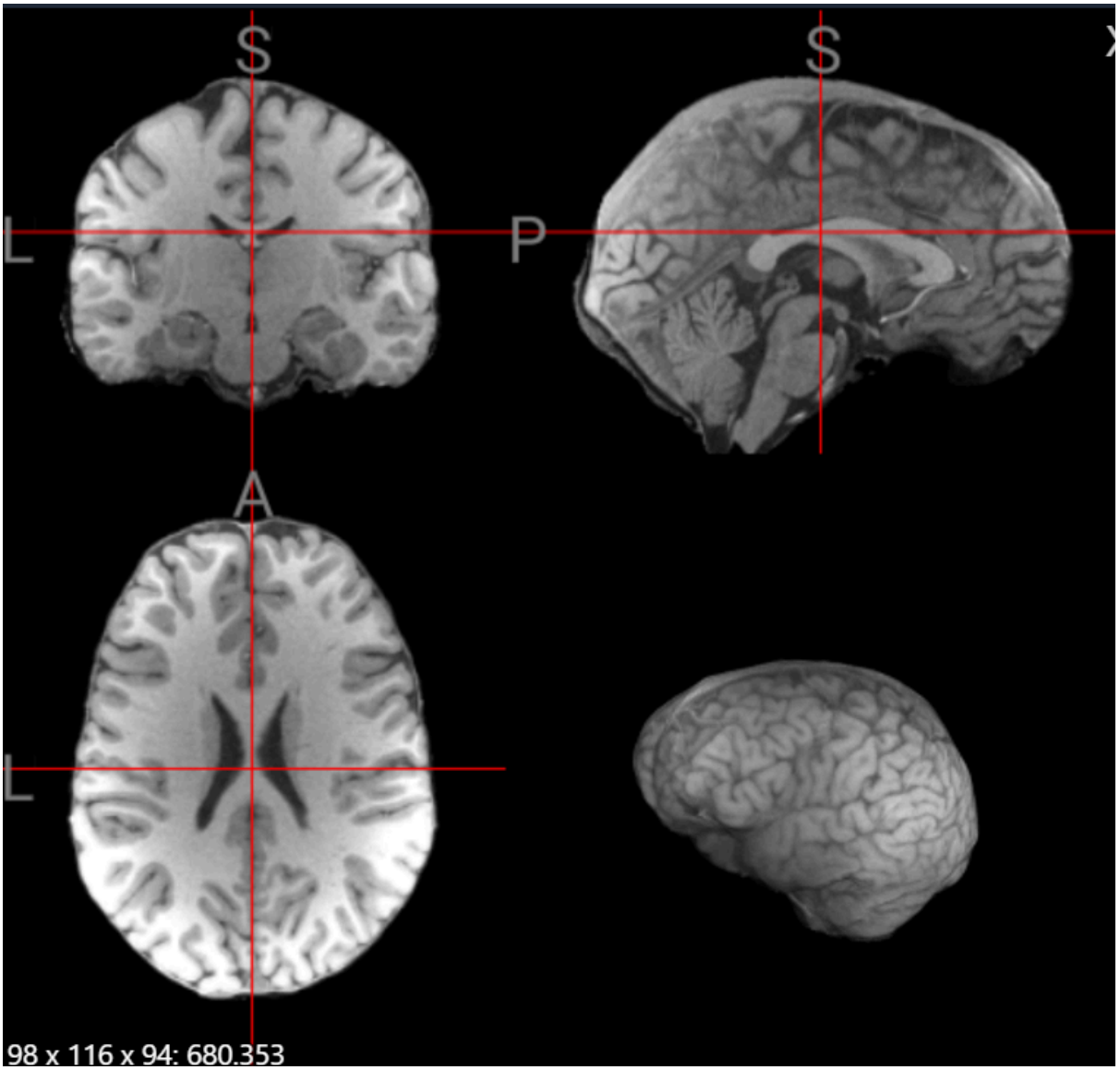
Here, we can see that the deformed registration does a better job at processing the image than the affine. We can see better the central part of the brain, called the lateral ventricles (butterfly shape):



atlas_segmented:

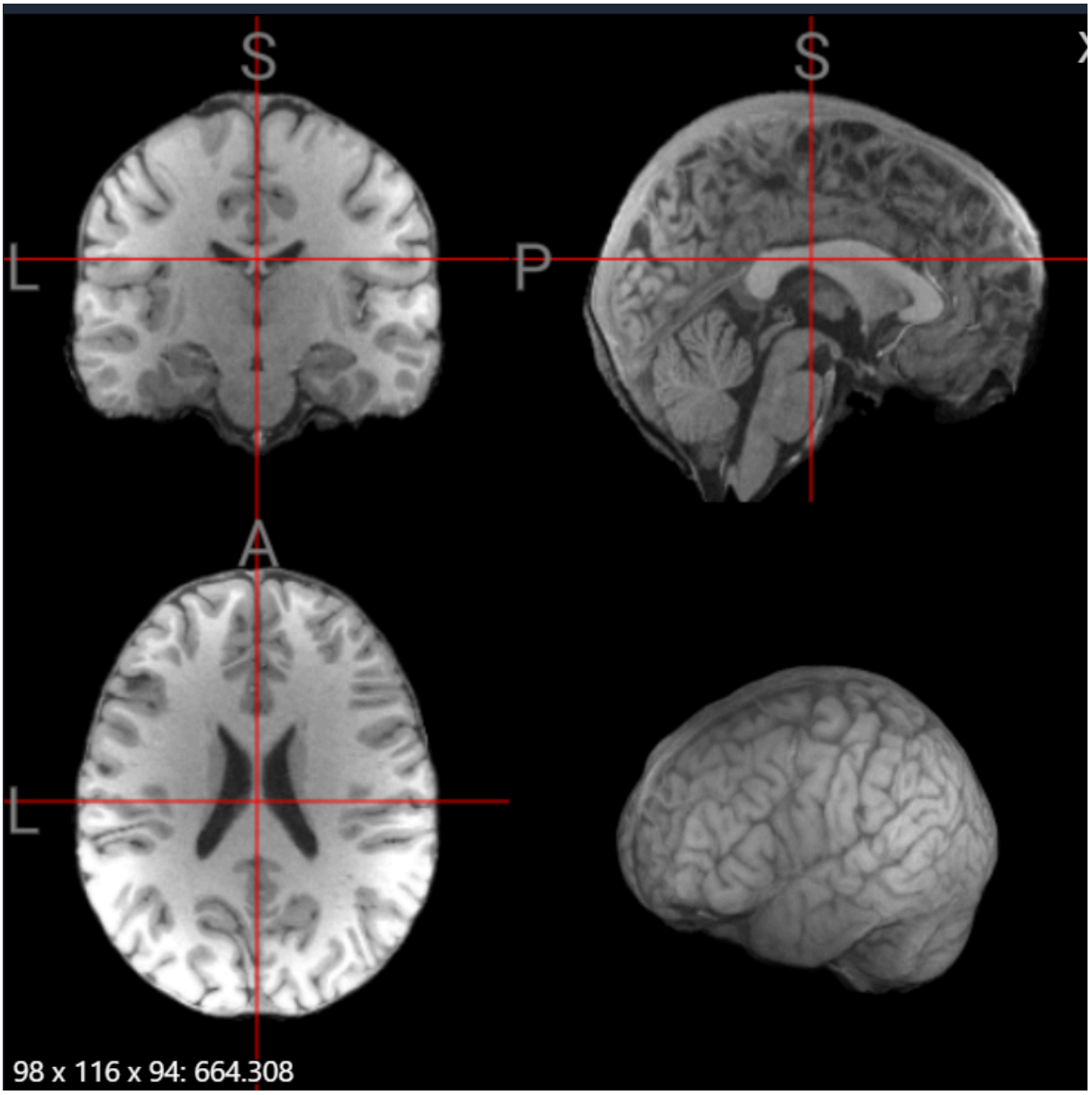


segmented_affine_patient:



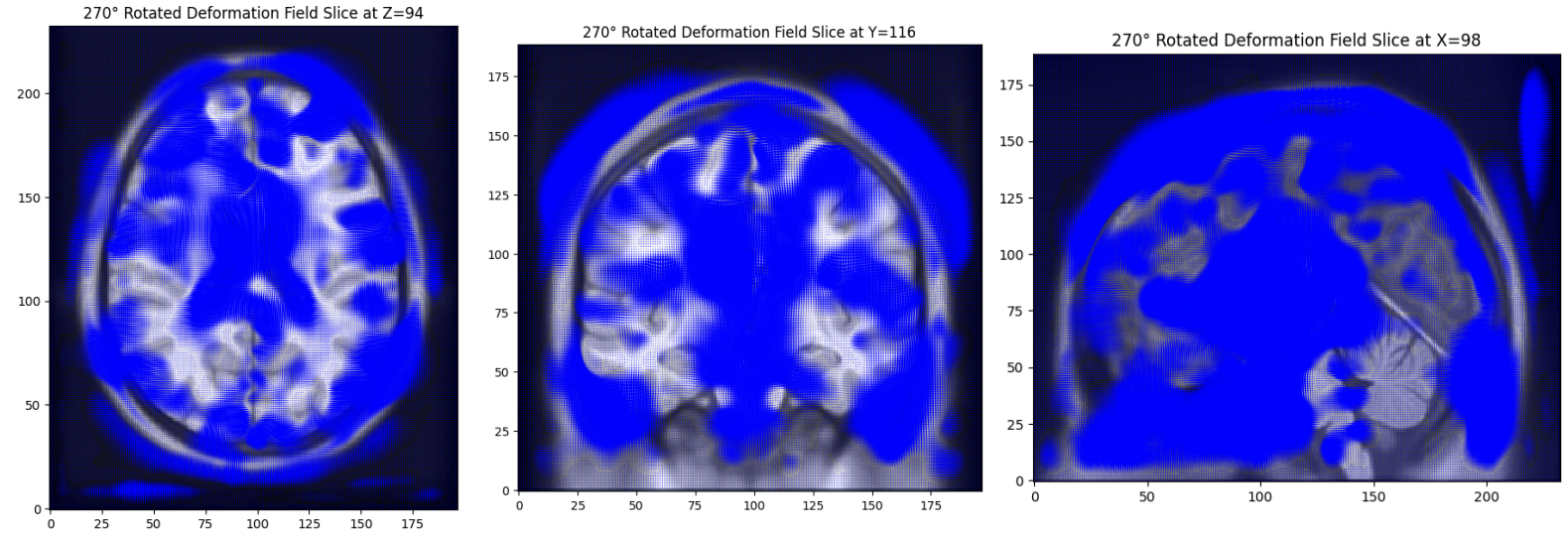
segmented_deformed_patient:

Here, again, we can see that the deformed registration shows best the central part of the brain and that makes it more similar to the segmented atlas image:.

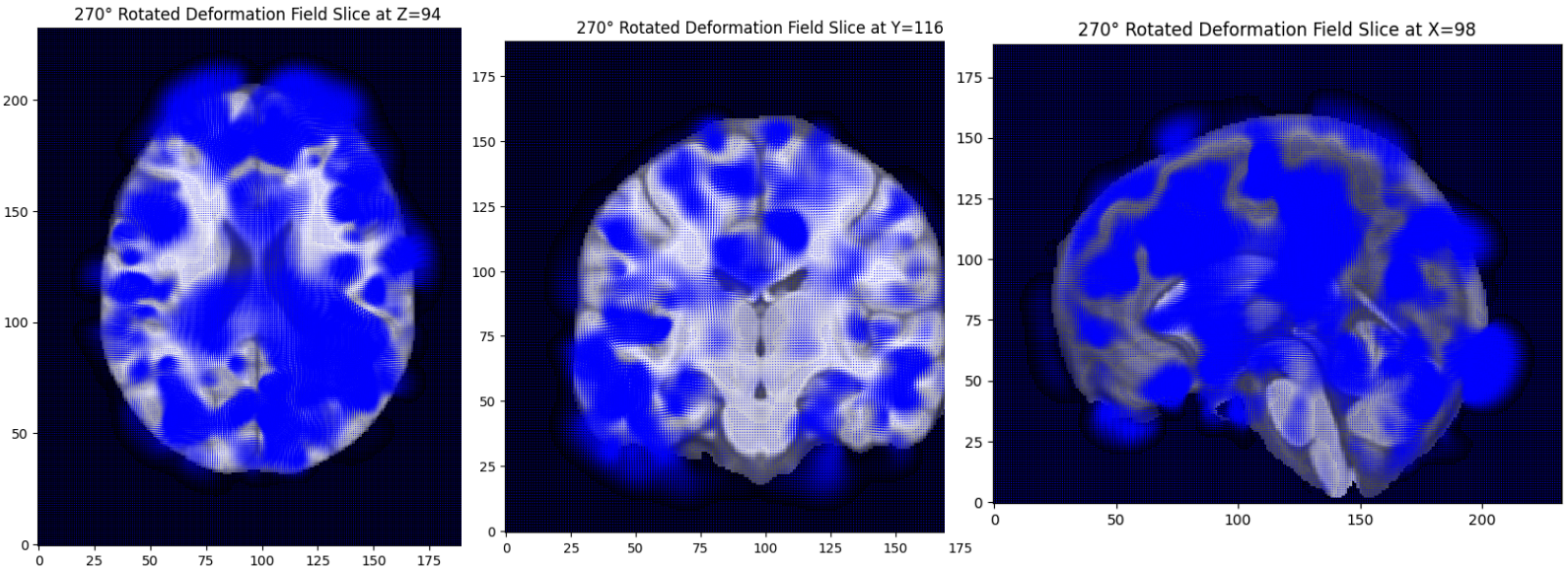


• **Visualize (and store in .png) the two 3D deformation fields generated by the deformable registration (one for the segmented subject, one for the non-segmented subject). You can plot the fields in two dimensions, as they are 3D**

non-segmented patient deformation field:



segmented patient deformation field:



6. Report- Answer these questions:

• Compare the alignment quality achieved by affine and deformable registration. Which do you think that performs better?

In 3D, **alignment quality** depends on the ability of the registration to align the **voxels** of the moving volume to those of the reference volume. Affine registration applies **global transformations**, aligning large structures effectively but failing to account for local mismatches. On the other hand, deformable registration uses **non-linear transformations** to align smaller, localized features, leading to **better voxel-level alignment** (as we can see in the transformed images). Deformable registration typically performs better due to its flexibility in handling intricate anatomical differences.

How do you think this performance can be measured? Put two examples on metrics to see how well registration has been done.

To evaluate this in 3D, metrics such as **Mutual Information (MI)**, which measures statistical dependency between voxel intensities, and the **Dice Similarity Coefficient (DSC)**, which quantifies voxel-level overlap between segmented regions, are commonly used. High MI or DSC values indicate precise alignment at the voxel level, reflecting accurate registration in 3D space:

```python
# %% Performance Metrics for Quantitative Comparison
from sklearn.metrics import mean_squared_error
from scipy.spatial.distance import dice

# Reload registered images to ensure proper variable definitions
affine_registered_img = nib.load("segmented_affine_patient.nii")
affine_registered_data = affine_registered_img.get_fdata()

deformed_registered_img = nib.load("segmented_deformed_patient.nii")
deformed_registered_data = deformed_registered_img.get_fdata()

# Flatten the data for metrics calculation
template_flat = template_data.ravel()
affine_flat = affine_registered_data.ravel()
deformable_flat = deformed_registered_data.ravel()

# Mean Squared Error (MSE)
mse_affine = mean_squared_error(template_flat, affine_flat)
mse_deformable = mean_squared_error(template_flat, deformable_flat)

# Dice Similarity Coefficient (requires binary masks)
template_binary = (template_flat > 0).astype(int)
affine_binary = (affine_flat > 0).astype(int)
deformable_binary = (deformable_flat > 0).astype(int)

dice_affine = 1 - dice(template_binary, affine_binary)
dice_deformable = 1 - dice(template_binary, deformable_binary)
```

The performance metrics for the segmented images indicate **Dice Similarity Coefficients** of **0.91** for affine and **0.9841** for deformable, showing **slightly better overlap** for deformable registration.

Moreover, the MSE values are **84557.47** for affine and **103722.98** for deformable. Since lower MSE values indicate better pixel-wise alignment, the higher MSE for deformable registration suggests that **affine registration actually achieves better overall pixel-level alignment**. However, deformable registration **excels at capturing local structural variations**, which may explain the higher MSE due to **more complex adjustments in the image.**

For the non-segmented images, the performance metrics indicate poorer alignment than the segmented case, possibly because deformable registration can struggle without clear structural boundaries to guide the transformation.

• Compare the dimensions for the affine registration object and the deformation field object. Do they have the same shape? Why?

For example, in the non-segmented registrations:

```
# Compare the dimensions of the affine registration object and the deformation field object

# Affine registration output: transformed volume shape
affine_transformed_shape = transformed.shape
print(f"Shape of the affine-transformed volume: {affine_transformed_shape}")

# Deformation field: forward mapping field dimensions
deformation_field_shape = deformation_field.shape
print(f"Shape of the deformation field: {deformation_field_shape}")
```

[41] ✓ 0.0s

```
Shape of the affine-transformed volume: (197, 233, 189)
Shape of the deformation field: (197, 233, 189, 3)
```

The shape of the affine-transformed volume **(197, 233, 189)** indicates the spatial resolution of the 3D image after an affine transformation. The three parameters correspond to the **number of voxels along the x-axis (197, left-to-right), y-axis (233, anterior-to-posterior), and z-axis (189, inferior-to-superior)**. These dimensions represent the resampled image's size in each spatial direction, aligned globally with the template image through linear transformations such as translation, scaling, rotation, or shearing.

The deformation field shape **(197, 233, 189, 3)** includes an additional dimension. The first three parameters are identical to those of the affine-transformed volume, representing the spatial resolution of the image. The fourth parameter **(3)** introduces a **displacement vector for each voxel**, with components corresponding to the x, y, and z axes. These vectors describe **how individual voxels must move to align precisely with the template image.**

• **You performed registration using both segmented and non-segmented volumes. Which aligns better the brain volumes?**

Segmented volumes generally yield better registration for specific regions of interest, like the brain, because **non-relevant structures** (e.g., the skull, background noise) are **excluded**, reducing interference during alignment.

Non-segmented volumes include additional structures that can affect registration accuracy by introducing noise or **misalignments**.

- **Imagine a hospital provided you two raw brain MRIs from the same patient in different timestamps. Describe a pipeline on how to achieve perfect registration of 2 brain MRIs, including all processes that you think would be interesting to perform:**

To register two brain MRIs from the same patient taken at different times, I'd start with **preprocessing**. This involves normalizing the images to reduce scanner-related variations, removing **non-relevant tissues** like the skull, and **resampling** both images to the same resolution and voxel dimensions. This ensures they are clean and ready for alignment.

Next is **affine registration**, which will use **mutual information** (for different imaging setups) or **normalized cross-correlation** (if the images are similar), to check how well the images overlap. An **optimization algorithm**, such as **gradient descent**, then adjusts the alignment step-by-step by changing parameters like translation, rotation, scaling, and shearing until the images align well on the 3D space.

Once the **global alignment** is done, I'd move to **deformable registration** to fine-tune the alignment for smaller, **local** differences. This step uses flexible models, like **B-splines** or **diffeomorphic transformations**, which allow specific parts of the brain to stretch or **warp** as needed to better match the two images. A similarity measure checks how well these local adjustments improve the alignment. To ensure the warping stays smooth and realistic, a **regularization process** limits overly sharp or unnatural changes.

The process is often done in multiple stages, starting with a lower resolution version of the images for broad adjustments, then refining the alignment on higher resolution versions for **more detailed accuracy**. This **multi-resolution** approach helps make the process more **efficient and precise**.

Finally, I'd evaluate the results visually by overlaying the images and quantitatively with metrics like the **Dice Similarity Coefficient**. Once satisfied with the results, I'd apply the transformations to get a final, accurately registered MRI.